Parte 2 – Demo en vivo en AWS

Video Pedidos360 · Evaluación 2 (DSY1107) · Sebastián · va entre la Parte 1 y la Parte 3

Tu parte debe durar como máximo 3:00. Las partes de tus compañeros ya suman 6:59 (Parte 1: 2:29 · Parte 3: 4:30) y la pauta permite hasta 10 minutos. Apunta a 2:45: di cada idea en una frase y no esperes en silencio a que carguen las páginas (déjalas abiertas antes).

Antes de grabar

Frontend	https://6iqkstpi9c.execute-api.us-east-1.amazonaws.com
Consola RabbitMQ	http://100.30.255.138:15672 · usuario y clave en `infra/aws/.env.cloud` (solo abre desde tu IP)
Decodificar el token	https://jwt.ms

☐ **Entra ID:** Pedidos360-Web → Authentication → "+ Agregar URI de redireccionamiento" → Aplicación de página única → `https://6iqkstpi9c.execute-api.us-east-1.amazonaws.com` (sin barra final). La URL anterior ya no existe.

☐ Inicia sesión una vez en el frontend y crea un pedido de prueba. Si da 403, asigna el rol Cliente en Enterprise applications → Pedidos360-API → Users and groups.

☐ Deja estas pestañas abiertas, en este orden: Entra ID (Información general) · EC2 · RDS · API Gateway `pedidos360-api` → Routes · frontend (con sesión iniciada) · jwt.ms · consola RabbitMQ (con sesión iniciada).

☐ Abre la terminal de la nube: EC2 → `pedidos360-ec2` → Conectar → Session Manager → Conectar.

☐ Abre PowerShell con este comando ya escrito (sin Enter): `curl.exe -i https://6iqkstpi9c.execute-api.us-east-1.amazonaws.com/api/pedidos`

☐ Las DLQ están en 0. Si haces pruebas antes y quedan mensajes: consola RabbitMQ → cada `*.dlq` → Purge Messages.

☐ Graba antes de las 17:52 hora de Chile (13:52 hora del lab, fin de la sesión). Si el lab se reinicia, la IP y el gateway cambian.

Tomas (2:50 en total)

1. Entra ID · 15 s `req. 5`

Información general (Id. de inquilino) → Usuarios (2) → Registros de aplicaciones (Pedidos360-API y Pedidos360-Web).

> "Este es nuestro tenant de Microsoft Entra ID, con dos usuarios y las aplicaciones del frontend y de la API, que define los roles Cliente y Admin."

2. Infraestructura en AWS · 25 s req. 1, 2 y 7

EC2 `pedidos360-ec2` en ejecución → RDS `pedidos360-db` disponible → API Gateway `pedidos360-api` → Routes (las rutas `/api/...` con JWT) → Authorization (autorizador `entra-jwt`).

> "En AWS tenemos la EC2 con RabbitMQ y los tres microservicios, la base de datos en RDS y el API Gateway como API Manager: cada ruta tiene un autorizador JWT con el emisor y la audiencia de Entra ID."

3. Login y JWT · 25 s req. 6

Frontend → Iniciar sesión (o muestra que ya estás dentro) → F12 → Network → petición `pedidos` → copia el header `Authorization` sin "Bearer" → pégalo en jwt.ms.

> "El frontend inicia sesión con OpenID Connect y PKCE. Entra ID entrega este JWT: emisor nuestro tenant, audiencia la API y el rol Cliente."

4. Pedido a través del API Manager · 15 s req. 3 y 4

Crea un pedido (Ana · Café 1 kg · 15990) → aparece en la tabla; en Network el POST va a `6iqkstpi9c.execute-api...` con 200.

> "El frontend llama al backend a través del API Gateway; con token válido la petición pasa y el pedido se guarda."

5. Rechazo sin token · 10 s req. 4

En PowerShell, Enter al comando ya escrito → `HTTP/1.1 401 Unauthorized`.

> "Sin un token válido, el API Gateway responde 401 y la petición no llega al backend."

6. docker compose en la nube · 15 s req. 11

En Session Manager:

```
cd /opt/pedidos360 && sudo docker compose ps
```

> "Todo corre con docker compose en la EC2: los dos nodos de RabbitMQ y los tres microservicios."

7. RabbitMQ: clúster, exchanges y colas · 30 s req. 8, 10 y 9

Overview → tabla Nodes (`rabbit@rabbit1` y `rabbit@rabbit2`) → Exchanges (`pedidos.events.exchange` topic, `pedidos.tasks.exchange` direct, `pedidos.dlx`) → Queues (3 colas + 3 DLQ, todas en 0).

> "RabbitMQ corre en un clúster de dos nodos. Tenemos un exchange topic para eventos, uno direct para comandos y el DLX. Cada una de las tres colas tiene su DLQ, con retención de 24 horas."

8. Una DLQ funcionando en vivo · 30 s req. 9

En el frontend crea un pedido con descripción `Prueba #FAIL-TICKET` → en Session Manager:

```
sudo docker logs --tail 6 pedidos360-procesador
→ intento 1/3 ... intento 2/3 ... NACK -> DLQ
```

→ en RabbitMQ, Queues: `tickets.dlq` con 1 mensaje.

> "Este pedido simula que el servicio de tickets está caído: el procesador reintenta tres veces y lo rechaza, y el mensaje queda en la DLQ de tickets. La notificación y el documento del mismo pedido sí se procesaron."

9. Traspaso · 5 s

> "Ahora [nombre del Integrante 3] explica cómo está hecho esto en el código."

Orden final del video

Orden	Parte	Archivo	Duración
1	Parte 1 – Integrante 2 (arquitectura)	WhatsApp Video 2026-10-23 at 11.20.04 PM.mp4	2:29
2	Parte 2 – Sebastián (demo en AWS)	tu grabación	≤ 3:00
3	Parte 3 – Integrante 3 (código)	WhatsApp Video 2026-10-23 at 11.04.13 PM.mp4	4:30

Únelos en Clipchamp en ese orden y exporta en 1080p. Revisa que el total no pase de 10:00.